

Crêpier psychorigide

Introduction (5')

Retour sur le cours précédent sur devine mon nombre:

- Rappel de l'algorithme de résolution,
- lien avec recherche dichotomique, par exemple dans un dictionnaire,
- très rapidement la complexité en $\log_2(N)$ en montrant démo pour $N = 1e30$

Explication règles (5')

- On veut ordonner la pile
- On ne peut que retourner avec une spatule

Activité débranchée(20')

Par groupes de 3 :

- 1) Ordonner 2-3 piles
- 2) 1 robot, 1 qui controle le robot à l'aveugle, 1 qui vérifie (5')
- 3) Proposer un algorithme (5')

Puis on met en commun, chaque groupe propose son algorithme, on essaie de formaliser un algorithme commun et d'identifier les instructions nécessaires : Position crepe plus grosse, Retourner, Tas trié, etc. (10')

Activité branchée (50')

On a comme instructions à disposition :

- `taille(n)` # Taille de la n-ème crêpe en partant du haut
- `retourner(n)` # Retourner sous la n-ème crêpe en partant du haut

et on aurait besoin de `tasTrié()` et `plusGrande(n)`.

Introduction au return (10')

Par groupes de 2-3, la moitié des groupes programme la fonction `tasTrié()` et l'autre moitié `plusGrande(n)`

Implémentation de fonction (15')

`tasTrié()` :

- 1) Commencer par afficher la taille de toutes les crepes avec une boucle for.
- 2) *Si possible faire trouver par élèves* : Au début, on part du principe que la pile est triée et on change si on voit une crêpe pas triée. Créer variable booléenne qui est True si la pile est triée et False sinon.

- 3) Qu'est-ce qu'on va vérifier à chaque itération de la boucle ? *Attention à réduire le range à N-1*
- 4) On retourne la variable booléenne

plusGrande(nombreCrepes) :

- 1) Commencer par afficher la taille de toutes les crepes avec une boucle for.
- 2) On va d'abord chercher la plus grande de toute la liste. *Si possible faire trouver par élèves : variable qui stocke la position de la plus grande vue jusqu'ici*
- 3) Qu'est-ce qu'on va vérifier à chaque itération de la boucle ?
- 4) On retourne la variable booléenne
- 5) On modifie la fonction pour ajouter un paramètre **nombreCrepes** qui spécifie parmi combien de crêpes on cherche la plus grande,

Implémentation de l'algorithme (25')

Les élèves forment des binômes complémentaires.

- Mise en commun et explication des fonctions (10')
- Implémentation de l'algorithme (10')

Conclusion (10')

- Correction au tableau de l'algo
- Discussion sur nombre d'étapes max

Canevas élève

```
# Import librairie
from crepes import init, taille, retourner
N = 7
init(N)

# Instructions disponibles :
taille(3) # Taille de la 3e crêpe en partant du haut
retourner(3) # Retourner sous la 3e crêpe en partant du haut

# Fonctions nécessaires

# Solution algorithme
```

Corrigé

```
# Import librairie
from crepes import init, taille, retourner

N = 7
init(N)

# Instructions disponibles :
taille(3) # Taille de la 3e crêpe en partant du haut
retourner(3) # Retourner sous la 3e crêpe en partant du haut

# Fonctions nécessaires
def tasTrié():
    estTrié = True
    for i in range(N-1):
        if taille(i) > taille(i+1):
            estTrié = False
    return estTrié

def plusGrande(nombreCrepes=N):
    idx = 0
    for i in range(nombreCrepes):
        if taille(i) > taille(idx):
            idx = i
    return idx

# Solution algorithme
idx = N
while tasTrié() == False:

    # On retourne sous la plus grande non triée
    tasDeCrepes = retourner(plusGrande(idx))

    # On retourne sous la 1ère non triée
    tasDeCrepes = retourner(idx-1)

    idx = idx - 1
```